

KẾ HOẠCH & LỘ TRÌNH CÁC GIAI ĐOẠN PHÁT TRIỂN DỰ ÁN VAHIZTECH

Hệ Sinh Thái SaaS Multi-Tenant với Java (Spring Boot, Quarkus) & Next.js

1. TỔNG QUAN DỰ ÁN & MỤC TIÊU CHIẾN LƯỢC

1.1. Bối cảnh & Tầm nhìn

Vahiztech là hệ sinh thái phần mềm dạng dịch vụ (**B2B & B2C SaaS**) hướng kiến trúc đa người thuê (**Multi-Tenancy**). Hệ thống cung cấp nền tảng quản trị định danh và truy cập tập trung (**Central IAM & SSO**), tích hợp các sản phẩm số chuyên biệt:

- **Next.js Portal (`nextjs-portal`)**: Cổng thông tin trung tâm cho người dùng doanh nghiệp (Tenant) và cá nhân, hỗ trợ phân giải subdomain động, quản trị giấy phép và điều hướng sản phẩm.
- **CourseDemy (`coursedemy-client`)**: Nền tảng E-Learning chuyên nghiệp (quản lý khóa học, video streaming, bài tập, tiến độ học viên).
- **VihoTask (`vihotask-client`)**: Hệ thống quản lý công việc và quy trình dự án (Kanban, Scrum, sprint tracking, giao việc và báo cáo).
- **Core Admin Service (`backend-admin-client`)**: Dịch vụ quản trị tổ chức, cấp phát tenant và đồng bộ người dùng với Keycloak Admin API.

1.2. Mục tiêu Kỹ thuật Cốt lõi

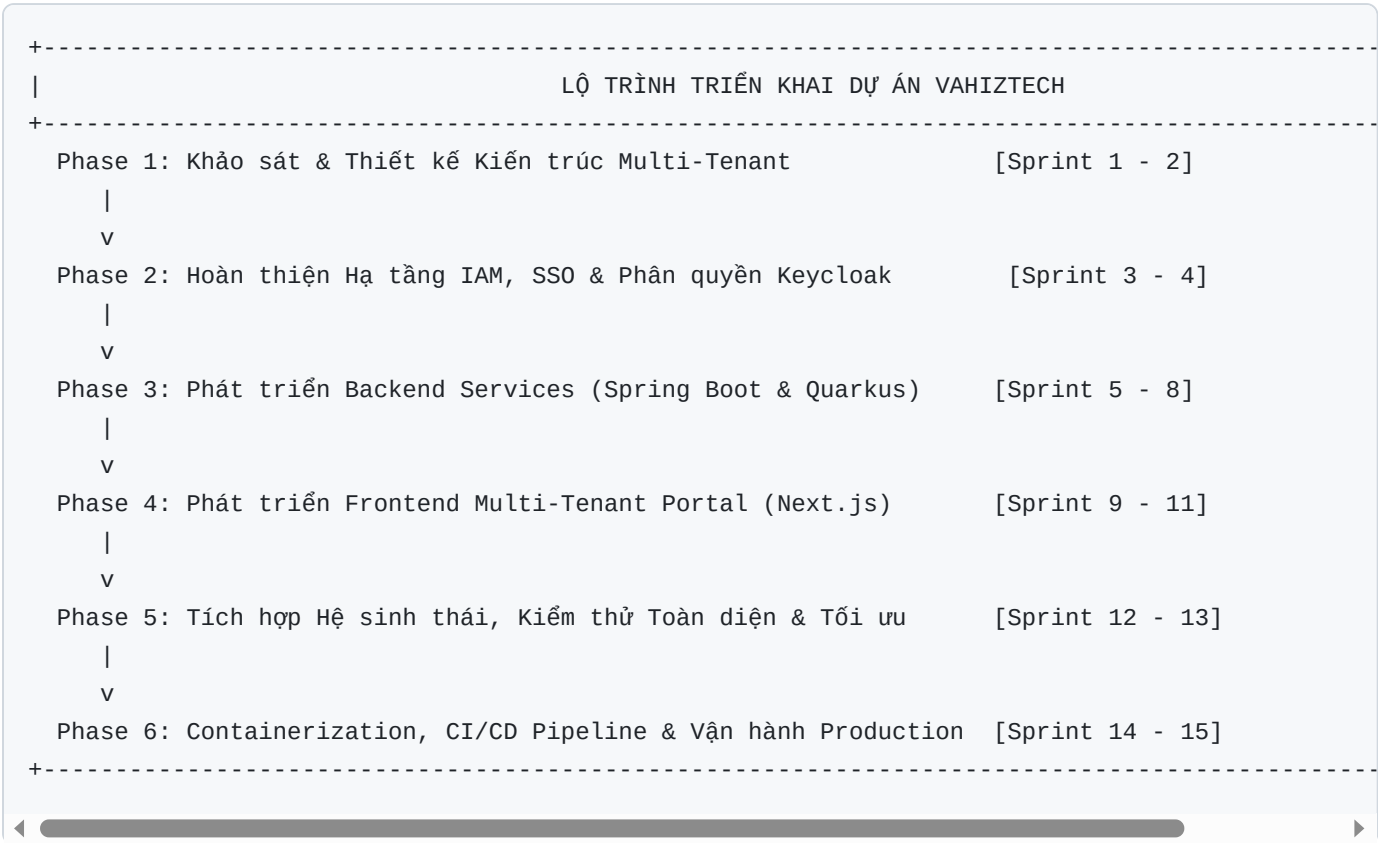
1. **Phân lập Dữ liệu Tuyệt đối (Strict Tenant Isolation)**: Đảm bảo không xảy ra rò rỉ dữ liệu chéo giữa các Tenant (tổ chức) ở cả tầng API, bộ nhớ đệm (Cache) và Cơ sở dữ liệu (Database).
2. **Xác thực Đơn điểm (Single Sign-On - SSO)**: Người dùng chỉ cần đăng nhập một lần thông qua Keycloak 25+ và có thể truy cập mượt mà vào tất cả các sản phẩm được cấp phép bản quyền (`licenses`).
3. **Kiến trúc Tối ưu Hiệu năng (High-Performance Polyglot Java)**:
 - **Spring Boot 3.3+**: Xử lý logic nghiệp vụ phức tạp (Core Domains), giao dịch ACID toàn vẹn, bảo vệ dữ liệu với Spring Data JPA & Hibernate.
 - **Quarkus 3.14+ (GraalVM Native)**: Đảm nhiệm API Gateway / BFF, xử lý luồng sự kiện thời gian thực (Kafka, SSE, WebSockets), khởi động tức thì (< 50ms) và tiêu tốn tối thiểu tài nguyên (RAM < 35MB).
 - **Next.js 14+ (App Router)**: Giao diện SSR/SSG hiện đại, tối ưu SEO, bảo mật phiên làm việc với OIDC PKCE Flow.

4. **Sẵn sàng Mở rộng (Cloud-Native Scalability):** Hỗ trợ đóng gói container hóa Docker, dễ dàng triển khai trên Kubernetes và mở rộng theo chiều ngang (Horizontal Pod Autoscaling).
-

2. PHÂN ĐỊNH VAI TRÒ CÔNG NGHỆ (TECH-STACK MATRIX)

Thành phần	Công nghệ chính	Trách nhiệm & Vai trò trong Hệ thống	Giao thức Giao tiếp
Central IAM & SSO	Keycloak 25+, PostgreSQL 16 Alpine	- Quản lý Danh tính tập trung (IdP), User, Tenant Organization. - Cấp phát Token JWT nhúng claims: <code>tenant_id</code>, <code>licenses</code>, <code>roles</code>. - Hỗ trợ OIDC, PKCE S256, OAuth2 Client Credentials.	OIDC / HTTPS / JWKS
Frontend Portal	Next.js 14+ (App Router), TypeScript, TailwindCSS, NextAuth.js	- Cổng đăng nhập SSO tập trung cho khách hàng. - Phân giải Tenant theo Subdomain (<code>tenant-a.vahiztech.com</code>) hoặc Slug. - Ẩn/hiện tính năng động theo <code>licenses</code> và <code>roles</code>.	HTTPS / REST / SSE
Core Resource Server	Java 21, Spring Boot 3.3+, Spring Security 6, Spring Data JPA	- Xử lý nghiệp vụ lõi của CourseDemy & VihoTask. - Xác thực chữ ký JWT qua Keycloak JWKS. - Phân lập dữ liệu Tenant tự động bằng Hibernate Filter. - Giao tiếp Keycloak Admin API quản lý User/Org.	REST / JSON / JDBC
BFF & Real-time Gateway	Java 21, Quarkus 3.14+, SmallRye JWT, Mutiny Reactive, Kafka	- API Gateway gom cụm dữ liệu (Aggregation BFF) cho Portal. - Xử lý luồng thông báo thời gian thực (Real-time Notification via SSE). - Tiếp nhận và phân phối sự kiện bất đồng bộ qua Apache Kafka. - Biên dịch GraalVM Native Image cho siêu hiệu năng.	Reactive REST / SSE / Kafka
Message Broker	Apache Kafka / Redpanda	- Hàng đợi truyền thông điệp bất đồng bộ giữa các Microservices. - Đồng bộ trạng thái Tenant, User Provisioning, Audit Logs.	Kafka Protocol (TCP)
Database Tier	PostgreSQL 16 (Multi-Schema / Partitioning), Redis Cache	- Lưu trữ dữ liệu hệ thống và cấu hình Keycloak. - Redis lưu trữ Session cache và Distributed Lock.	TCP / SQL

3. LỘ TRÌNH CHI TIẾT 6 GIAI ĐOẠN PHÁT TRIỂN



GIAI ĐOẠN 1: KHẢO SÁT NGHIỆP VỤ, THIẾT KẾ KIẾN TRÚC & CHUẨN HÓA DỮ LIỆU

Mục tiêu: Thiết lập nền tảng kiến trúc vững chắc, chuẩn hóa mô hình dữ liệu đa người thuê và định nghĩa đặc tả giao tiếp API trước khi viết code. **Thời lượng dự kiến:** Sprint 1 - Sprint 2 (4 tuần).

Các công việc chi tiết:

1. Phân tích Yêu cầu Nghiệp vụ SaaS Multi-Tenant:

- Xác định quy trình đăng ký, kích hoạt và cấp phép tổ chức (Tenant Onboarding Lifecycle).
- Phân loại 3 nhóm đối tượng người dùng: Quản trị viên tổ chức (org_admin), thành viên tổ chức (org_member), người dùng tự do (individual_user).
- Xây dựng mô hình ma trận phân quyền (RBAC kết hợp ABAC) dựa trên roles và licenses (coursedemy , vihotask).

2. Thiết kế Mô hình Phân lập Dữ liệu Multi-Tenancy:

- Đánh giá và chốt phương án phân lập: Kết hợp Shared Database - Discriminator Column (tenant_id) cho dữ liệu dùng chung và Schema-per-Tenant cho khách hàng doanh nghiệp yêu cầu bảo mật cao.
- Thiết kế Entity Relationship Diagram (ERD) cho Domain CourseDemy và VihoTask.

3. Đặc tả Giao tiếp API & Hợp đồng Dữ liệu (API Contracts):

- Xây dựng tài liệu OpenAPI 3.0 (Swagger) cho các endpoint của CourseDemy, VihoTask và Quarkus BFF.
- Định dạng chuẩn cho HTTP Response: `{ success, data, error, timestamp, traceId }`.
- Thiết kế cấu trúc thông điệp sự kiện (Event Envelope Schema) trên Apache Kafka: `{ eventId, tenantId, eventType, timestamp, payload }`.

4. Chuẩn bị Môi trường Phát triển:

- Thiết lập chuẩn Coding Convention, SonarQube rules, Git Flow và quy định Merge Request cho Java & TypeScript.

Sản phẩm bàn giao (Deliverables):

- ☒ Tài liệu Kiến trúc Hệ thống (System Architecture Document - SAD).
- ☒ Bản vẽ thiết kế Database ERD và Chiến lược Phân lập Multi-Tenancy.
- ☒ Bộ file đặc tả OpenAPI Specification (`openapi.yaml`).
- ☒ Quy chuẩn Coding Standards & Git Workflow.

GIẢI ĐOẠN 2: HOÀN THIỆN HẠ TẦNG IAM, SSO & PHÂN QUYỀN KEYCLOAK

Mục tiêu: Thiết lập và vận hành ổn định hệ thống Keycloak 25+, tự động hóa cấu hình Realm, Clients, Protocol Mappers và kịch bản kiểm thử bảo mật. **Thời lượng dự kiến:** Sprint 3 - Sprint 4 (4 tuần).

Các công việc chi tiết:

1. Cấu hình & Tối ưu hóa Keycloak 25+:

- Vận hành cụm Keycloak kết hợp PostgreSQL 16 Alpine thông qua Docker Compose.
- Bật tính năng tổ chức (`--features=organization`) phục vụ quản lý phân cấp doanh nghiệp.
- Hoàn thiện cấu hình `realm-config.json` cho realm `ecosystem-realm`.

2. Cấu hình Chi tiết các Clients:

- `nextjs-portal` : Public Client, bật PKCE (Proof Key for Code Exchange) mã hóa S256, cấu hình chuẩn Redirect URIs và Web Origins tránh lỗi CORS.
- `coursedemy-client` : Bearer-only Resource Server, chỉ chấp nhận JWT token hợp lệ.
- `vihotask-client` : Bearer-only Resource Server, phân lập tài nguyên theo tenant.
- `backend-admin-client` : Confidential Client, cấp quyền Service Account (`manage-users` , `view-users`) để Core Service có thể quản trị user từ xa.

3. Xây dựng Protocol Mappers cho Custom Claims:

- Map thuộc tính người dùng `tenant_id` vào Access Token & ID Token (kiểu String).
- Map danh sách quyền sản phẩm `licenses` (kiểu Multivalued String JSON Array).
- Map danh sách Realm Roles (`roles`) vào Token Payload.

4. Tự động hóa Kiểm thử Cấp quyền (Automated Verification):

- Viết và hoàn thiện kịch bản `test-token.sh` kiểm thử lấy token trực tiếp từ Keycloak qua OAuth2 Password Grant và Client Credentials.
- Kiểm tra tính đúng đắn của token giải mã (Base64 JWT payload) với đầy đủ claims.

Sản phẩm bàn giao (Deliverables):

- ☒ Hạ tầng Keycloak + PostgreSQL hoạt động ổn định trên Docker.
- ☒ File `realm-config.json` đầy đủ clients, mappers, roles và pre-seeded users.
- ☒ Script `test-token.sh` kiểm thử tự động đạt 100% kịch bản kiểm tra Token Claims.

GIẢI ĐOẠN 3: PHÁT TRIỂN BACKEND SERVICES BẰNG JAVA (SPRING BOOT & QUARKUS)

Mục tiêu: Xây dựng toàn bộ các dịch vụ backend, hiện thực hóa nghiệp vụ kinh doanh với Spring Boot và các dịch vụ cổng giao tiếp thời gian thực hiệu năng cao với Quarkus. **Thời lượng dự kiến:** Sprint 5 - Sprint 8 (8 tuần).

Nhánh 1: Spring Boot Microservices (Resource Servers)

1. Module Bảo mật & Xác thực Token:

- Tích hợp `spring-boot-starter-oauth2-resource-server`.
- Cấu hình xác thực chữ ký JWT dựa trên JWKS URL của Keycloak.
- Xây dựng `CustomJwtAuthenticationConverter` chuyển đổi claims (`roles`, `licenses`, `tenant_id`) thành Spring `GrantedAuthority`.

2. Hiện thực Cơ chế Phân lập Dữ liệu Tự động (Multi-Tenant Context):

- Xây dựng `TenantContextHolder` (sử dụng `ThreadLocal` hoặc Scoped Bean) để lưu trữ `tenant_id` của request hiện hành.
- Tích hợp Hibernate Filter (`@Filter` & `@FilterDef`) tự động thêm điều kiện `WHERE tenant_id = :tenantId` vào tất cả các truy vấn JPA.
- Tạo `@PrePersist` Entity Listener tự động gán `tenant_id` khi tạo bản ghi mới.

3. Phát triển Core Business Domains:

- **CourseDemy Service:** Quản lý khóa học, danh mục, bài giảng video, đăng ký học tập, theo dõi tiến độ hoàn thành.
- **VihoTask Service:** Quản lý dự án, bảng công việc Kanban, danh sách Task, phân công thành viên, trạng thái tiến độ.
- **Admin Management Service:** Tích hợp Keycloak Admin Client REST API để tự động tạo Tenant, thêm User vào Tổ chức khi có giao dịch đăng ký mới.

4. Bảo vệ Endpoint bằng Phương thức Phân quyền (Method Security):

- Áp dụng `@PreAuthorize` kết hợp kiểm tra Role và License:
`@PreAuthorize("hasRole('org_admin') and hasAuthority('LICENSE_coursedemy')")`.

Nhánh 2: Quarkus Microservices (BFF & High-Throughput / Reactive)

1. Xây dựng Quarkus API Gateway / BFF:

- Sử dụng RESTEasy Reactive và Mutiny để xử lý non-blocking I/O.
- Tích hợp `quarkus-oidc` và `quarkus-smallrye-jwt` kiểm tra token với độ trễ cực thấp (< 2ms).
- Gom cụm dữ liệu (Aggregation): 1 request từ Portal gọi vào Quarkus BFF sẽ được phân tán lấy dữ liệu đồng thời từ cả CourseDemy và VihoTask rồi phản hồi về Client.

2. Notification & Real-time Event Streaming Hub:

- Tích hợp Apache Kafka với `quarkus-messaging-kafka`.
- Lắng nghe các sự kiện: `TaskAssignedEvent`, `CourseCompletedEvent`, `TenantProvisionedEvent`.
- Cung cấp kênh Server-Sent Events (SSE) `/api/v1/notifications/stream` đẩy thông báo tức thời về giao diện Next.js Portal.

3. Tối ưu Hóa Biên Dịch GraalVM Native Image:

- Cấu hình `application.properties` và Dockerfile Native.
- Kiểm thử chỉ số tài nguyên: Bộ nhớ tiêu thụ dưới 35MB RAM, thời gian khởi động lạnh < 50ms.

Sản phẩm bàn giao (Deliverables):

- ☒ Mã nguồn hoàn chỉnh các Service Spring Boot (CourseDemy, VihoTask, Admin).
- ☒ Mã nguồn Quarkus BFF Gateway & Real-time Notification Engine.
- ☒ Bộ Unit Test & Mock Server kiểm tra xác thực quyền truy cập đạt độ phủ code (Coverage) > 80%.

GIẢI ĐOẠN 4: PHÁT TRIỂN FRONTEND MULTI-TENANT PORTAL VỚI NEXT.JS

Mục tiêu: Xây dựng giao diện trung tâm trực quan, hiện đại, hỗ trợ SSO mượt mà, phân giải tenant theo tên miền và hiển thị tính năng động theo bản quyền. **Thời lượng dự kiến:** Sprint 9 - Sprint 11 (6 tuần).

Các công việc chi tiết:

1. Khởi tạo Kiến trúc Next.js App Router:

- Cấu trúc thư mục chuẩn: `src/app`, `src/components`, `src/lib`, `src/hooks`, `src/types`.
- Cấu hình Tailwind CSS, Shadcn UI / Radix Primitives cho hệ thống Design System đồng nhất.

2. Tích hợp Keycloak OIDC Authentication với PKCE:

- Cấu hình NextAuth.js (`auth.ts` / NextAuth v5) kết nối Keycloak Provider.
- Luồng Authorization Code Flow kèm PKCE S256 bảo mật phía client.
- Tự động đồng bộ claims `tenant_id`, `licenses`, `roles` vào NextAuth Session Token.
- Xây dựng cơ chế Silent Token Refresh khi Access Token hết hạn (3600s).

3. Phát triển Multi-Tenant Edge Middleware:

- Viết `middleware.ts` bắt mọi request tại Edge:
 - Tách hostname để nhận diện Subdomain (ví dụ: `alpha.vahiztech.com` -> `tenant_id = tenant-alpha`).
 - Kiểm tra tính hợp lệ của người dùng đối với Tenant mục tiêu.
 - Bảo vệ các tuyến đường yêu cầu xác thực (`/dashboard/*`, `/settings/*`).

4. Phát triển Giao diện & Điều khiển Phân quyền (License & Role Guard UI):

- Xây dựng Component bọc quyền: `<LicenseGuard license="coursedemy">...</LicenseGuard>`.
- Nếu tài khoản có license `coursedemy`: Hiển thị phân hệ Học tập CourseDemy.
- Nếu tài khoản có license `vihotask`: Hiển thị phân hệ Quản lý công việc VihoTask.
- Tích hợp Client SSE lắng nghe sự kiện thông báo thời gian thực từ Quarkus Hub.

Sản phẩm bàn giao (Deliverables):

- ☒ Giao diện Next.js Portal hoàn chỉnh, tương thích Responsive trên Desktop và Mobile.
- ☒ Luồng đăng nhập SSO Keycloak PKCE mượt mà, hỗ trợ Tenant Routing.
- ☒ Tích hợp trọn vẹn API gọi dữ liệu từ Spring Boot và luồng thông báo Quarkus SSE.

GIẢI ĐOẠN 5: TÍCH HỢP HỆ THỐNG, KIỂM THỬ TOÀN DIỆN & TỐI ƯU HIỆU NĂNG

Mục tiêu: Kết nối toàn bộ các thành phần rời rạc thành một thể thống nhất, kiểm thử chịu tải, kiểm thử bảo mật thâm nhập và tối ưu hóa tài nguyên. **Thời lượng dự kiến:** Sprint 12 - Sprint 13 (4 tuần).

Các công việc chi tiết:

1. Kiểm thử Tích hợp Đầu-Cuối (End-to-End Testing):

- Thiết lập kịch bản Playwright / Cypress kiểm thử luồng người dùng: Đăng nhập SSO -> Vào Portal -> Chuyển hướng CourseDemy -> Tạo task VihoTask -> Nhận thông báo Real-time.
- Tự động hóa kiểm thử Backend với Testcontainers (PostgreSQL + Keycloak container thực tế).

2. Kiểm thử Bảo mật Thâm nhập (Security Penetration Testing):

- **Tenant Data Isolation Test:** Giả lập người dùng của `tenant-alpha` gửi request can thiệp ID của `tenant-beta` -> Bắt buộc hệ thống phải trả về HTTP 403 / 404.
- **License Tampering Test:** Thử nghiệm giả mạo token không có license `coursedemy` truy cập API video -> Bắt buộc từ chối truy cập.
- Quét lỗ hổng OWASP Top 10, SQL Injection, CSRF và XSS.

3. Kiểm thử Hiệu năng & Chịu tải (Load & Stress Testing):

- Sử dụng công cụ k6 / Apache JMeter giả lập 5,000 - 20,000 người dùng đồng thời (Concurrent Users).
- Đo lường và đối sánh chỉ số TPS (Transactions Per Second) và độ trễ (Latency P95, P99) giữa Spring Boot và Quarkus Native.

- Tối ưu hóa Database Connection Pool (HikariCP cho Spring Boot, Agroal cho Quarkus).

Sản phẩm bàn giao (Deliverables):

- ☒ Báo cáo kiểm thử tự động E2E và độ phủ Integration Test.
- ☒ Báo cáo bảo mật (Security Audit Report) xác nhận không có lỗ hổng rò rỉ dữ liệu Tenant.
- ☒ Báo cáo hiệu năng chịu tải hệ thống (Performance Benchmark Report).

GIẢI ĐOẠN 6: CONTAINERIZATION, CI/CD PIPELINE & VẬN HÀNH PRODUCTION

Mục tiêu: Đóng gói các ứng dụng thành container chuẩn, xây dựng pipeline tự động kiểm tra và triển khai mã nguồn lên môi trường Production an toàn, tin cậy. **Thời lượng dự kiến:** Sprint 14 - Sprint 15 (4 tuần).

Các công việc chi tiết:

1. Chuẩn hóa Container Images (Multi-stage Dockerfiles):

- Spring Boot: Sử dụng Eclipse Temurin 21 JRE Distrosless / Layered Jar để tối ưu dung lượng image (< 200MB).
- Quarkus: Build GraalVM Native Executable chạy trên base image Scratch/Ubi-minimal (< 50MB).
- Next.js: Tận dụng cơ chế `output: 'standalone'` trong `next.config.js` để tạo image siêu nhẹ (< 120MB).

2. Xây dựng Pipeline CI/CD (GitHub Actions):

- **Continuous Integration (CI):** Tự động chạy Unit Test, Linting, SonarQube Scanner khi có Pull Request.
- **Continuous Delivery (CD):** Tự động build Docker Image, đánh tag phiên bản ngữ nghĩa (Semantic Versioning), đẩy lên Container Registry (Docker Hub / AWS ECR) và triển khai tự động qua GitOps (ArgoCD / Helm).

3. Thiết lập Hạ tầng Giám sát Toàn diện (Observability Stack):

- Thu thập Metrics: Cấu hình Prometheus cào chỉ số từ Actuator (Spring Boot) và MicroProfile Metrics (Quarkus).
- Trực quan hóa: Thiết lập Dashboard Grafana theo dõi CPU, RAM, JVM Heap, Request Count, Error Rate 5xx.
- Distributed Tracing: Tích hợp OpenTelemetry và Jaeger theo dõi dấu vết request xuyên suốt từ Next.js -> Quarkus BFF -> Spring Boot.
- Quản lý Log tập trung: Thu thập log qua FluentBit đẩy về Elasticsearch / Grafana Loki.

4. Xây dựng Kế hoạch Sao lưu & Khôi phục Sau Thảm họa (Backup & Disaster Recovery):

- Script tự động backup Database PostgreSQL định kỳ mỗi ngày lưu trữ trên S3.
- Quy trình khôi phục Keycloak Realm và dữ liệu Tenant khi có sự cố.

Sản phẩm bàn giao (Deliverables):

- ☒ Bộ file Dockerfile và `docker-compose.prod.yml` hoàn chỉnh.
 - ☒ Kịch bản GitHub Actions CI/CD Pipeline hoạt động tự động.
 - ☒ Cụm giám sát Prometheus + Grafana với các bảng điều khiển trực quan.
 - ☒ Tài liệu Hướng dẫn Vận hành & Khôi phục Thảm họa (Runbook & DR Guide).
-

4. MA TRẬN PHÂN BỐ TRÁCH NHIỆM (RACI MATRIX)

Nhiệm vụ / Hạng mục công việc	Project Manager	Solution Architect	Java Backend Dev (Spring/Quarkus)	Frontend Dev (Next.js)	DevOps / SecOps	QA / QC Engineer
Phân tích nghiệp vụ & Thiết kế kiến trúc	A	R	C	C	C	I
Cấu hình Keycloak IAM, Realm & Mappers	I	A	R	C	R	I
Xây dựng Spring Boot Resource Servers	I	A	R	I	C	C
Xây dựng Quarkus BFF & Notification Hub	I	A	R	C	C	C
Phát triển Next.js Multi-Tenant Portal	I	A	C	R	C	C
Tích hợp SSO PKCE & Inter- service API	I	A	R	R	I	C
Kiểm thử Tích hợp, E2E & Bảo mật	I	A	C	C	C	R
Xây dựng Docker, CI/CD Pipeline & Monitoring	I	A	C	C	R	I
Triển khai Production & Chuyển giao	A	C	I	I	R	I

Quy ước RACI:

- **R (Responsible):** Người trực tiếp thực thi công việc.
- **A (Accountable):** Người chịu trách nhiệm cuối cùng về kết quả.
- **C (Consulted):** Người được tham vấn ý kiến chuyên môn.
- **I (Informed):** Người được thông báo kết quả.

5. TIÊU CHUẨN NGHIỆM THU (DEFINITION OF DONE - DoD)

Một giai đoạn hoặc tính năng chỉ được xem là hoàn thành khi đáp ứng đủ các điều kiện khắt khe sau:

1. **Chức năng (Functionality):** Hoàn thành 100% các tiêu chí chấp nhận (Acceptance Criteria) được mô tả trong tài liệu đặc tả.
2. **Chất lượng Mã nguồn (Code Quality):**
 - Vượt qua kiểm tra Linting và Static Analysis, không có lỗi Critical / Blocker trên SonarQube.
 - Code Coverage của Unit Tests và Integration Tests đạt tối thiểu **80%**.
3. **Bảo mật Multi-Tenant (Security Compliance):**
 - 100% API bảo mật bắt buộc phải kiểm tra chữ ký JWT và phân lập chính xác `tenant_id`.
 - Không xuất hiện bất kỳ cảnh báo rò rỉ dữ liệu Tenant nào trong quá trình kiểm thử thâm nhập.
4. **Hiệu năng (Performance SLA):**
 - Thời gian phản hồi trung bình (API Response Time) của Quarkus BFF < 50ms và Spring Boot < 200ms với 95% số lượng requests (P95).
5. **Đóng gói & Triển khai (CI/CD):**
 - Docker build thành công không lỗi, pass toàn bộ automated test pipeline trên CI.
6. **Tài liệu hóa (Documentation):**
 - Cập nhật tài liệu API trên Swagger/OpenAPI và bổ sung hướng dẫn chạy/vận hành liên quan.

6. QUẢN TRỊ RỦI RO & PHƯƠNG ÁN DỰ PHÒNG

Rủi ro tiềm ẩn	Mức độ	Biện pháp Phòng ngừa & Xử lý
Rò rỉ dữ liệu chéo giữa các Tenant	Rất cao	- Bắt buộc áp dụng Hibernate Filter tự động ở tầng ORM. - Viết bộ kiểm thử tự động giả lập truy vấn chéo tenant chạy định kỳ trên CI. - Thiết lập kiểm tra kép (Double-check) tại tầng Service và tầng DB.
Nghẽn cổ chai tại Keycloak khi tải cao	Trung bình	- Tăng thời gian sống của Access Token lên 1 giờ để giảm tần suất gọi cấp token. - Microservices xác thực JWT offline thông qua JWKS caching công khai, không gọi trực tiếp Keycloak mỗi request. - Cụm Keycloak chạy tối thiểu 2 nodes cluster phía sau Load Balancer.
Lỗi tương thích GraalVM Native Image	Trung bình	- Sử dụng các thư viện chính thức nằm trong Quarkus Universe (đã hỗ trợ GraalVM out-of-the-box). - Luôn kiểm thử song song cả bản JVM build và Native build trên môi trường Staging.
Độ trễ đồng bộ dữ liệu người dùng/tổ chức	Thấp	- Sử dụng Apache Kafka làm hàng đợi thông điệp đảm bảo xử lý ít nhất một lần (At-least-once delivery) kèm cơ chế Retry / Dead Letter Queue (DLQ).